

后端应用抽象响应实体

如果接口返回的内容格式不规范，会对前端开发人员造成困扰，我们来对接口做一些优化，使用统一的返回对象来使每次请求后返回的结果更加规范。

定义返回体

```
package org.opengoofy.congomall.springboot.starter.convention.result;

import lombok.Data;
import lombok.experimental.Accessors;

import java.io.Serializable;

/**
 * 全局返回对象
 */
@Data
@Accessors(chain = true)
public class Result<T> implements Serializable {

    private static final long serialVersionUID = 5679018624309023727L;

    /**
     * 正确返回码
     */
    public static final String SUCCESS_CODE = "0";

    /**
     * 返回码
     */
    private String code;

    /**
     * 返回消息
     */
    private String message;
```

```

/**
 * 响应数据
 */
private T data;

/**
 * 请求ID
 */
private String requestId;

public boolean isSuccess() {
    return SUCCESS_CODE.equals(code);
}
}

```

同时，为了方便 Result 的创建，通过一个 Results 工具类用来创建返回类。

```

package org.opengoofy.congomall.springboot.starter.web;

import org.apache.skywalking.apm.toolkit.trace.TraceContext;
import org.opengoofy.congomall.springboot.starter.convention.errorcode.BaseErrorCode;
import org.opengoofy.congomall.springboot.starter.convention.exception.AbstractException;
import org.opengoofy.congomall.springboot.starter.convention.result.Result;

import java.util.Optional;

/**
 * 全局返回对象构造器
 */
public final class Results {

    /**
     * 构造成功响应
     *
     * @return
     */
    public static Result<Void> success() {
        return new Result<Void>()
            .setCode(Result.SUCCESS_CODE)
            .setRequestId(TraceContext.traceId());
    }
}

```

```

/**
 * 构造带返回数据的成功响应
 *
 * @param data
 * @param <T>
 * @return
 */
public static <T> Result<T> success(T data) {
    return new Result<T>()
        .setCode(Result.SUCCESS_CODE)
        .setRequestId(TraceContext.traceId())
        .setData(data);
}

/**
 * 构建服务端失败响应
 *
 * @return
 */
protected static Result<Void> failure() {
    return new Result<Void>()
        .setCode(BaseErrorCode.SERVICE_ERROR.code())
        .setRequestId(TraceContext.traceId())
        .setMessage(BaseErrorCode.SERVICE_ERROR.message());
}

/**
 * 通过 {@link AbstractException} 构建失败响应
 *
 * @param abstractException
 * @return
 */
protected static Result<Void> failure(AbstractException abstractException) {
    String errorCode = Optional.ofNullable(abstractException.getErrorCode())
        .orElse(BaseErrorCode.SERVICE_ERROR.code());
    String errorMessage = Optional.ofNullable(abstractException.getErrorMessage())
        .orElse(BaseErrorCode.SERVICE_ERROR.message());
    return new Result<Void>()
        .setCode(errorCode)
        .setRequestId(TraceContext.traceId())

```

```

        .setMessage(errorMessage);
    }

    /**
     * 通过 errorCode、errorMessage 构建失败响应
     *
     * @param errorCode
     * @param errorMessage
     * @return
     */
    protected static Result<Void> failure(String errorCode, String errorMessage) {
        return new Result<Void>()
            .setCode(errorCode)
            .setRequestId(TraceContext.traceId())
            .setMessage(errorMessage);
    }
}

```

文章中部分 API 可能会让大家比较陌生，这里给出两份源码地址。

- `org.opengoofy.index12306.framework.starter.convention.result.Result`
- `org.opengoofy.index12306.framework.starter.web.Results`

有什么好处

统一返回 Result 有什么好处呢？

回答这个问题，首先我们先看下 Result 中都包含哪几个参数。

1. 返回码

code，这是统一返回体的核心字段，用来标记该请求执行返回结果。

如果返回正常，大部分网站接口返回的往往是 0。另外，根据阿里 Java 开发手册（崇山版）错误码列表规定，返回 00000，这里以前者为主。

返回码不止是正确的，同样包含错误。关于错误异常码的定义放到下篇文章详细说明。

2. 返回消息

message, 当前请求状态码描述, 如果接口请求成功, 为空。如果接口请求异常, 这里是对应的异常错误提示信息。

3. 返回数据

data, 当前请求返回响应数据。

4. 请求 ID

requestId, 该 ID 也可以当作链路追踪 TraceID 来看, 用来串联当前请求的上下文, 以及分布式链路追踪错误排查的依据。

通过统一返回体约束, 可以很好与前端形成良好规范, 避免多个项目 “各自为战”, 不方便重构和监控。

真实请求

我们真实访问下商品服务根据 SKU ID 获取商品详情的接口, 看看它的返回体, 很清晰是不是。

```
{
  "code": "0",
  "message": null,
  "data": {
    "productBrand": {
      "id": 1477055818267025408,
      "name": "小米",
      "desc": "小米是一家以手机、智能硬件和IoT平台为核心的互联网公司，以智能手机、智能电视、笔记本等丰富的产品与服务。致力于让全球每个人都能享受科技带来的美好生活",
      "pic": null,
      "sort": 1
    },
    "productSpu": {
      "id": 1477055850256982016,
      "categoryId": 1477055818388660224,
      "brandId": 1477055818267025408,
      "name": "小米MIX Fold2 轻薄折叠 骁龙8+旗舰处理器 徕卡光学镜头 自研微水滴形态转轴 12GB+512GB 月影黑 5G手机",

```

```
"productSn": "100033890205",
"pic": "",
"photoAlbum": null,
"price": 9999,
"promotionPrice": 9988,
"promotionStartTime": "2022-09-14T13:02:26.000+00:00",
"promotionEndTime": "2022-09-14T13:02:26.000+00:00",
"subTitle": "轻薄折叠机,整机轻至262g,展开厚度5.4mm,骁龙8+旗舰处理器!【点击抢购小米
11Ultra】",
"sales": 0,
"unit": "件",
"detail": null,
"publishStatus": 0,
"newStatus": 0,
"recommandStatus": 0
},
"productSkus": [
{
  "id": 1477056250724933632,
  "productId": 1477055850256982016,
  "price": 9999,
  "stock": 99526,
  "lockStock": 474,
  "pic": "",
  "attribute": "[{"key": "外观", "value": "月影黑"}, {"key": "版本",
\, "value": "12GB+256GB"}]"
},
{
  "id": 1477056250741710848,
  "productId": 1477055850256982016,
  "price": 11999,
  "stock": 299,
  "lockStock": 5,
  "pic": "",
  "attribute": "[{"key": "外观", "value": "月影黑"}, {"key": "版本",
\, "value": "12GB+512GB"}]"
},
{
  "id": 1477056338104868864,
  "productId": 1477055850256982016,
  "price": 9999,
```

```
    "stock": 188,
    "lockStock": 2,
    "pic": "",
    "attribute": "[{"key": "外观", "value": "星耀金"}, {"key": "版本", "value": "12GB+256GB"}]"
  },
  {
    "id": 1477056338335555584,
    "productId": 1477055850256982016,
    "price": 11999,
    "stock": 299,
    "lockStock": 4,
    "pic": "",
    "attribute": "[{"key": "外观", "value": "星耀金"}, {"key": "版本", "value": "12GB+512GB"}]"
  }
]
},
"requestId": "4a2c99691f1f4d7dbc944ddbcf60108d.423.16777712688418813",
"success": true
}
```

更新: 2023-07-17 14:23:24

原文: <<https://www.yuque.com/magestack/12306/xdcxfcfn6exgef3z>>